# IA Aplicada com LLMs

## Aula 15 — Observabilidade

# Roteiro

1. **Por que uma convenção** — a árvore de *spans* e como ela é exportada
2. **As quatro perguntas** — latência, custo, término, erro por ferramenta
3. **Por onde decompor** — ferramenta, versão, agente
4. **O alerta** — e a degradação silenciosa
5. **O que o painel não diz** — e a avaliação online, com o juiz da aula de *evals*

# Objetivos

Ao final deste deck, espera-se que o aluno seja capaz de:

- **instrumentar** o agente nas convenções semânticas GenAI do OpenTelemetry;
- **definir** as quatro métricas, **lê-las** no painel da plataforma que recebe os *spans* e **decompô-las** por ferramenta, versão e agente;
- **definir** um alerta com limiar justificado;
- **distinguir** o que o painel mede do que ele não mede — o acerto — e **amostrar** o tráfego para medi-lo com o juiz da aula de *evals*.

# Parte 1

## Por que uma convenção

# A árvore de spans

```
invoke_agent  "auditor-de-despesas"                    <- raiz
├── chat  "mistral-small-latest"
│     gen_ai.request.model         = "mistral-small-latest"
│     gen_ai.usage.input_tokens    = 1842
│     gen_ai.usage.output_tokens   = 96
├── execute_tool  "consultar_politica"
│     gen_ai.tool.name             = "consultar_politica"
├── execute_tool  "buscar_regulamento"
└── chat  "mistral-small-latest"
```

Convenções semânticas **GenAI do OpenTelemetry**.

## O ganho não é estético: é trocar de ferramenta sem reinstrumentar

O custo de **não poder trocar** apareceu duas vezes nas aulas anteriores:

```
frameworks   a API do framework, movida em 12 meses
MCP          a especificação do protocolo, revista 5 vezes em 20 meses
```

## Um segundo ganho, mais relevante em equipe: a convenção OBRIGA A NOMEAR

```
gen_ai.usage.input_tokens   significa o mesmo em todo lugar

tokens                      significa o que o autor quis
                            e ninguém mais sabe se inclui a saída
```

## Antes e depois

```
# ANTES — formato próprio, legível só pela equipe que o definiu
{"passo": 3, "ferramenta": "consultar_politica",
 "argumentos": {"categoria": "refeicao"}, "tokens": 1938}

# DEPOIS — convenção
with tracer.start_as_current_span("execute_tool") as span:
    span.set_attribute("gen_ai.tool.name", "consultar_politica")
    span.set_attribute("gen_ai.operation.name", "execute_tool")
```

**O dado é o mesmo.** O que mudou foi quem consegue lê-lo sem documentação da equipe.

# Exportar: OTLP, em lote, para a plataforma

```
agente ──spans + resource──▶ POST <endpoint>/v1/traces ──▶ Grafana Cloud
                             http/protobuf, em lote, em segundo plano
```

- o **resource** identifica quem emitiu: serviço e versão implantada;
- destino e credencial em variáveis de ambiente (`OTEL_EXPORTER_OTLP_*`);
- o código **não cita a plataforma**: trocá-la é trocar a configuração.

# Parte 2

## As quatro perguntas

## As métricas a que o plano do auditor chega

**Não é lista universal**: outra arquitetura chega a outra lista.

- **Latência p50 e p95** — a média é inútil: a distribuição tem **cauda longa**, porque o número de passos varia. O p95 é o que o usuário insatisfeito sentiu.
- **Custo por execução** — a unidade que a aula de escolha de modelos estabeleceu como a compreendida pela gestão.
- **Distribuição dos motivos de término.**
- **Taxa de erro por ferramenta.**

Nenhuma é nova. Todas saem do *trace* que já existia.

## A terceira quase nenhum painel comercial tem

E a razão é simples: **quase ninguém modela término como dado**.

O término foi modelado como dado em arquitetura de agentes — e por isso pode ser medido.

```
motivos de término:
    respondeu ............ 78%
    orcamento_esgotado ... 19%        <- degradação SILENCIOSA
    laco_detectado ........  3%
```

## A quarta: erros que não derrubam nada

Erros de **domínio** não derrubam a execução — voltam como observação e o agente contorna.

```
taxa de erro por ferramenta:
    buscar_regulamento ... 22%        <- integração degradando
    consultar_politica ....  1%
```

Por isso não aparecem como falha, e **a única forma de vê-los é contá-los**.

# Parte 3

## Por onde decompor

## As métricas agregadas dizem QUE piorou. Não dizem ONDE.

Três cortes resolvem, e os três **já existem como dado**:

| Corte | Responde |
|---|---|
| **por ferramenta** | qual integração está falhando |
| **por versão** | a piora começou com qual mudança |
| **por agente** | qual participante errou |

## Por versão — obrigatório

```
                        v1.4      v1.5     delta
custo/execução       R$0,031   R$0,047    +52%
p95                     22 s      38 s     +73%
orcamento_esgotado        6%        19%    +13 p.p.
```

A v1.5 acrescentou uma ferramenta.

> Um painel agregado mostraria uma **média entre as duas versões**, e o efeito ficaria diluído. É o carimbo de *prompt engineering*, aplicado à operação — e a versão é a
> **tag da imagem**, gravada em todo *trace* por `VERSAO_SISTEMA`.

## Por agente — preparação para sistemas multiagente

Hoje há **um agente só**, e o corte tem um valor. O atributo é gravado desde já.

Num sistema multiagente, `invoke_agent` deixa de ser um *span* raiz único e passa a haver **um por participante** — e é essa decomposição que responde **"qual agente errou"** em minutos, em vez de dias.

Instrumentar o atributo agora custa uma linha; acrescentá-lo depois exige reinstrumentar.

# Parte 4

## O alerta

## Painel se olha. Alerta acorda.

A pergunta não é *"o que é interessante"*, e sim **o que justifica interromper alguém**.

**Custo por hora acima do teto** — direto, e o mais fácil de justificar. Um laço não detectado ou um pico de tráfego aparecem aqui **antes de aparecerem na fatura**.

## E o alerta que a maioria dos painéis não tem: `orcamento_esgotado` em alta

```
execuções que terminam por orçamento NÃO SÃO ERRO.
Devolvem 200, com uma resposta pior.
A taxa de erro NÃO SE MEXE.
A satisfação cai sem que nada acuse.
```

É o sintoma de **degradação silenciosa** — o corpus cresceu, o prompt ficou mais longo, uma ferramenta passou a devolver mais dado.

Sem essa métrica, o defeito só aparece **na reclamação**.

# Parte 5

## O que o painel não diz

## O painel mede COMO respondeu. Não mede SE ACERTOU.

```
execução e-8814
  termino ........ respondeu        <- saudável no painel
  latência ....... 4,1 s            <- saudável no painel
  custo .......... R$ 0,031         <- saudável no painel
  conclusão ...... "dentro da política"   <- ERRADA, e nada acusa
```

E o *prompt registry*, de *prompt engineering*, muda o prompt **sem *deploy***: a imagem é a mesma, a *tag* é a mesma, e o corte por versão **não separa o antes do depois**.

## A avaliação online, com a suíte de *evals*

*O serviço está acertando?* exige casos com resposta conhecida, critério de correção e juiz validado — **a suíte da aula de *evals***, agora sobre tráfego real.

| Da aula de *evals* | Em produção |
|---|---|
| o juiz validado contra rótulos humanos | julga uma **amostra** do tráfego do serviço |
| o *golden dataset* | recebe como caso cada falha encontrada na amostra |
| a suíte de regressão | compara versões antes de valerem — inclusive a do *registry* |

A taxa de acerto amostrada entra no painel **ao lado** das quatro métricas.